# Tasca S4.01. Creació de Base de Dades

Partint d'alguns arxius CSV dissenyaràs i crearàs la teva base de dades.

## Nivell 1

Descàrrega els arxius CSV, estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:

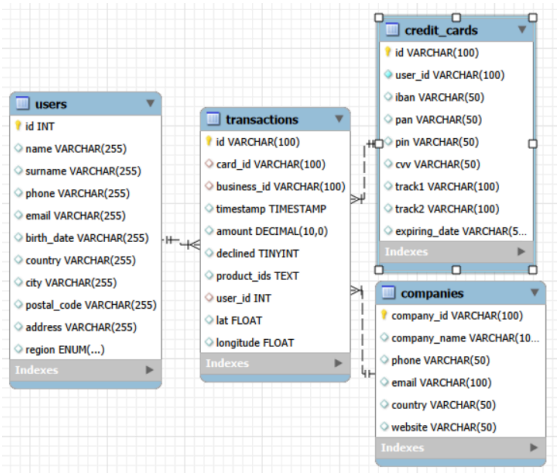
Revisant les dades, el primer que podem observar es que la majoria dels arxius tenen un index que ens permetrà establir una clau primària, facilitant la relació entre les diferents taules. En canvi, a la taula “transactions”, candidata principal a ser la taula de fets, hi haurem d’afegir un index autoincremental, i a més, haurem de fer la importació de dades tenint en compte que estan separades per “;” i no per “,” com tota la resta.

```sql
DROP DATABASE IF EXISTS sprint4;
CREATE DATABASE IF NOT EXISTS sprint4;

USE sprint4;

-- Creem una taula, on unificar els dos origen de dades d'usuaris

CREATE TABLE IF NOT EXISTS users (
id INT PRIMARY KEY NOT NULL,
name VARCHAR(255),
surname VARCHAR(255),
phone VARCHAR(255),
email VARCHAR(255),
birth_date VARCHAR(255),
country VARCHAR(255),
city VARCHAR(255),
postal_code VARCHAR(255),
address VARCHAR(255),
region ENUM('america', 'europa') -- creem un camp extra, per a difeneciar l'origen de les taules, amb dos possibles opcions
);

LOAD DATA INFILE "C:\\SQL_import\\american_users.csv"
INTO TABLE users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
IGNORE 1 ROWS
(id,name,surname,phone,email,birth_date,country,city,postal_code,address) -- especifiquem els camps, i establim el valor del camp extra
SET region = 'america';
```

```sql
LOAD DATA INFILE "C:\\SQL_import\\european_users.csv"
INTO TABLE users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
IGNORE 1 ROWS
(id,name,surname,phone,email,birth_date,country,city,postal_code,address)  -- especifiquem els camps, i establim el valor del camp extra
SET region = 'europa';

SELECT * FROM users;

-- creem i importem les dades de la taula companies
CREATE TABLE companies (
company_id VARCHAR(100) PRIMARY KEY NOT NULL,
company_name VARCHAR(100),
phone VARCHAR(50),
email VARCHAR(100),
country VARCHAR(50),
website VARCHAR(50)
);

LOAD DATA INFILE "C:\\SQL_import\\companies.csv"
INTO TABLE companies
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
IGNORE 1 ROWS;
```

```sql
-- creem i importem la taula transactions, relacionada amb les anteriors tres
CREATE TABLE transactions (
id VARCHAR(100) PRIMARY KEY NOT NULL,
card_id VARCHAR(100),
business_id VARCHAR(100),
timestamp TIMESTAMP,
amount DECIMAL,
declined TINYINT,
product_ids TEXT,
user_id INT,
lat FLOAT,
longitude FLOAT,
FOREIGN KEY (user_id) REFERENCES users(id),
FOREIGN KEY (card_id) REFERENCES credit_cards(id),
FOREIGN KEY (business_id) REFERENCES companies(company_id)
);

LOAD DATA INFILE "C:\\SQL_import\\transactions.csv"
INTO TABLE transactions
FIELDS TERMINATED BY ';'
IGNORE 1 ROWS;
```

```sql
-- creem i importem les dades de la taula credit_cards
CREATE TABLE credit_cards (
id VARCHAR(100) PRIMARY KEY NOT NULL,
user_id VARCHAR(100) NOT NULL,
iban VARCHAR(50),
pan VARCHAR(50),
pin VARCHAR(50),
cvv VARCHAR(50),
track1 VARCHAR(100),
track2 VARCHAR(100),
expiring_date VARCHAR(50)
);

LOAD DATA INFILE "C:\\SQL_import\\credit_cards.csv"
INTO TABLE credit_cards
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
IGNORE 1 ROWS;
```




Creem les taules, establint les relacions que hem trobat entre elles. Després, procedim a la importació de les dades des de els arxius .CSV.

# Exercici 1

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

```
104 •  SELECT *
105    FROM users u
106    WHERE u.id IN (
107        SELECT t.user_id
108        FROM transactions t
109        GROUP BY t.user_id
110        HAVING COUNT(*) > 80
111    );
112
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

| id | name | surname | phone | email | birth_date | country | city | postal_code | address | region |
|---|---|---|---|---|---|---|---|---|---|---|
| 185 | Molly | Gilliam | 0800 120 8023 | donec@outlook.couk | Dec 21, 1993 | United Kingdom | London | EC1A 1BB | P.O. Box 202, 5638 Mi Rd. | europa |
| 289 | Dxwgi | Hwcru | +98-309-8797 | dxwgi.hwcru@example.com | Aug 20, 1976 | Germany | Stuttgart | 70173 | 82 Hwcru Street | europa |
| 318 | Bnyr | Astuw | +33-120-9644 | bnyr.astuw@example.com | May 3, 1974 | Italy | Genoa | 16100 | 53 Astuw Street | europa |
| 454 | Sfzzoh | Xgvfridxs | +58-495-3945 | sfzzoh.xgvfridxs@example.com | Aug 28, 1962 | Poland | Gdansk | 80-001 | 52 Xgvfridxs Street | europa |
| NULL | NULL | NULL | NULL | NULL | NULL | NULL | NULL | NULL | NULL | NULL |

users 9 | Apply

Output

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 19 | 13:51:35 | SELECT * FROM transactions | 100000 row(s) returned |
| 20 | 13:51:57 | SELECT * FROM users u WHERE u.id IN ( SELECT t.user_id FROM transactions t GROUP BY t.... | 4 row(s) returned |

En aquest primer exercici, hem de fer una cerca relacionant les taules transaction i users. Farem una subconsulta per comptar quins usuaris tenen més de 80 transaccions, i sobre aquesta, relacionarem els ids dels usuaris de la taula principal.

# Exercici 2

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

```
16 •  SELECT company_name, ROUND(AVG(amount),2) average, iban FROM transactions t
17    JOIN credit_cards cc ON cc.id = t.card_id
18    JOIN companies co ON co.company_id = t.business_id
19    WHERE co.company_name = "Donec Ltd" AND declined = 0
20    GROUP BY iban;
21
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| company_name | average | iban |
|---|---|---|
| Donec Ltd | 356.33 | XX9114064011255863075868 05 |
| Donec Ltd | 143.00 | SK9446370242474562577506 |
| Donec Ltd | 257.00 | XX7767529178459529755556 40 |
| Donec Ltd | 140.00 | XX4138273622897193049089 90 |
| Donec Ltd | 240.00 | XX3477872460707696107803 08 |

Result 2

Output

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 10:38:39 | SELECT u.*, count(t.id) total_transactions FROM users u JOIN transactions t ON t.user_id = u.id GROUP B... | 4 row(s) returned |
| 2 | 10:40:57 | SELECT company_name, ROUND(AVG(amount),2) average, iban FROM transactions t JOIN credit_cards ... | 370 row(s) returned |

En aquest cas, relacionarem transactions amb credir_cards, establint les condicions del nom de la companyia i deixarem fora les transaccions no finalitzades, per ser més fidels en el resultat del promig.

# Nivell 2

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:

```sql
CREATE TABLE card_activity (
    id VARCHAR(100) PRIMARY KEY,
    estat_tarjeta VARCHAR (50)
)
AS
SELECT cc.id, user_id, iban,
    CASE
        WHEN SUM(ultimes.declined) = 3 THEN "tarjeta inactiva"
        ELSE "Tarjeta activa"
    END AS estat_tarjeta
FROM credit_cards cc
JOIN (SELECT t.card_id, t.declined,
        ROW_NUMBER() OVER (
            PARTITION BY t.card_id
            ORDER BY t.timestamp DESC
        ) AS numerades
    FROM transactions t) ultimes
    ON ultimes.card_id = cc.id
WHERE ultimes.numerades <= 3
GROUP BY cc.id, cc.user_id, cc.iban
ORDER BY cc.id;
;

SELECT * FROM card_activity;
ALTER TABLE card_activity
ADD FOREIGN KEY (id) REFERENCES credit_cards(id);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

| id | user_id | iban | estat_tarjeta |
|---|---|---|---|
| CcS-4857 | 276 | XX4857591835292505850771 | Tarjeta activa |
| CcS-4858 | 277 | XX8581768137002436094025 | Tarjeta activa |
| CcS-4859 | 278 | XX7826930491423553609370 | Tarjeta activa |
| CcS-4860 | 279 | XX5559590368835304645299 | Tarjeta activa |
| CcS-4861 | 280 | XX2035182877195191627307 | Tarjeta activa |

Result 29

Output

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 100 | 12:03:59 | SELECT * FROM card_activity | 5000 row(s) returned |
| 101 | 12:04:12 | SELECT cc.id, cc.user_id, cc.iban, estat_tarjeta FROM card_activity ca JOIN credit_cards cc ON cc.id =... | 4995 row(s) returned |

Per a crear la taula, farem servir la función ROW_NUMBER per a poder numerar les operacions. fent servir "l'índex" que hem creat, podem avaluar les tres primeres amb una subconsulta que ens torni només aquestes. Creant una columna a la taula amb CASE, comprovarem si les tres darreres operacions son declinades, comprovant si la suma de declinet resulta en 3.

```sql
ALTER TABLE card_activity
ADD FOREIGN KEY (id) REFERENCES credit_cards(id);
```

També crearem una FOREIGN KEY per a deixar la nova taula amb la taula pare, credit_cards.

# Exercici 1

Quantes targetes estan actives?

```sql
SELECT cc.id, user_id, iban, estat_tarjeta FROM card_activity ca
JOIN credit_cards cc ON cc.id = ca.id
WHERE estat_tarjeta = "tarjeta activa";
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

| id | user_id | iban | estat_tarjeta |
|---|---|---|---|
| CcS-4857 | 276 | XX4857591835292505850771 | Tarjeta activa |
| CcS-4858 | 277 | XX8581768137002436094025 | Tarjeta activa |
| CcS-4859 | 278 | XX7826930491423553609370 | Tarjeta activa |
| CcS-4860 | 279 | XX5559590368835304645299 | Tarjeta activa |
| CcS-4861 | 280 | XX2035182877195191627307 | Tarjeta activa |

Result 15

Output

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 53 | 13:23:00 | SELECT cc.id, user_id, iban, estat_tarjeta FROM card_activity ca JOIN credit_cards cc ON cc.id = ca.id ... | 4995 row(s) returned |
| 54 | 13:23:22 | SELECT cc.id, user_id, iban, estat_tarjeta FROM card_activity ca JOIN credit_cards cc ON cc.id = ca.id ... | 4995 row(s) returned |

Un cop creada la taula, només quedarà fer una cerca a la columna d'estat, per veure fàcilment quines son actives. El resultat retornat es 4995. Per tant, només 5 no han superat la criba. Afegim un JOIN a la taula pare, per obtenir dades addicionals de les targetes com l'usuari, IBAN.

# Nivell 3

Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades creada, tenint en compte que des de transaction tens product_ids. Genera la següent consulta:

El primer que farem serà crear la taula restant, amb informació dels productes.

```sql
CREATE TABLE products (
id VARCHAR(100) PRIMARY KEY NOT NULL,
product_name VARCHAR(100),
price VARCHAR (50),
colour VARCHAR(50),
weight VARCHAR(50),
warehouse_id VARCHAR(50)
);

LOAD DATA INFILE "C:\\SQL_import\\products.csv"
INTO TABLE products
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
IGNORE 1 ROWS;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

| id | product_name | price | colour | weight | warehouse_id |
|---|---|---|---|---|---|
| 1 | Direwolf Stannis | $161.11 | #7c7c7c | 1 | WH-4 |
| 10 | Karstark Dorne | $119.52 | #f4f4f4 | 2.4 | WH--5 |
| 100 | south duel | $40.43 | #6d6d6d | 3 | WH--95 |
| 11 | Karstark Dorne | $49.70 | #141414 | 2.7 | WH--6 |
| 12 | duel Direwolf | $181.60 | #a8a8a8 | 2.1 | WH--7 |

products 5

Output

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 10 | 11:20:55 | LOAD DATA INFILE "C:\\SQL_import\\products.csv" INTO TABLE products FIELDS TERMINATED BY... | 100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0 |
| 11 | 11:21:16 | SELECT * FROM products | 100 row(s) returned |

Un cop afegida la taula products, crearem una taula relacionada (comandes, productes), per a normalitzar les dades de la columna products_id, que conté les dades dels productes venuts. Per a aconseguir-ho, farem servir la funció JSON_TABLE, que ens permet convertir arrays en diverses rows, relacionades amb la id de la transacció. Abans però, haurem de normalitzar els arrais per convertir-los en una estructura JSON valida, concatenant els diferents valors, afegint “” a cada referencia, i encapsulant-los en unes altres “”, per donar el format “ “1”, “2”, “3”...

```sql
CREATE TABLE transaction_products (
    transaction_id VARCHAR (100),
    product_id VARCHAR (100),
    PRIMARY KEY (transaction_id, product_id),
    FOREIGN KEY (transaction_id) REFERENCES transactions(id),
    FOREIGN KEY (product_id) REFERENCES products(id)
);

INSERT INTO transaction_products (transaction_id, product_id)
SELECT
    t.id as transaction_id,
    p.id
FROM transactions t
JOIN JSON_TABLE(
        CONCAT('[', t.product_ids, ']'),
        '$[*]' COLUMNS (
            product_id INT PATH '$'
        )
    ) AS jt
JOIN products p
    ON p.id = jt.product_id;

SELECT * FROM transaction_products;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

| transaction_id | product_id |
|---|---|
| 001A60EA-DC9C-4E5A-9460-6628B100E7E1 | 1 |
| 0032F0BB-BBE6-4AA5-B5EE-EEAD533C0C48 | 1 |
| 00342381-503D-422D-85AB-F2D4FFAAD4C7 | 1 |
| 004C0A80-E537-46D8-BE44-343D2176DF15 | 1 |
| 004D1DB5-B2CB-4460-98B6-31C42CA96E5F | 1 |

transaction_products 39

Output

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 15793 | 13:15:31 | SELECT * FROM transaction_products | 253391 row(s) returned |

Ara ja tenim una taula que ens permet relacionar transaccions (amb les seves “id”) amb cada producte venut.

# Exercici 1

Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

```
112 ●  SELECT product_id, product_name, COUNT(product_id) AS total_ventas FROM transaction_products tp
113    JOIN products p ON id = product_id
114    GROUP BY product_id, product_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| product_id | product_name | total_ventas |
|---|---|---|
| 1 | Direwolf Stannis | 2468 |
| 10 | Karstark Dorne | 2571 |
| 100 | south duel | 2517 |
| 11 | Karstark Dorne | 2573 |
| 12 | duel Direwolf | 2558 |

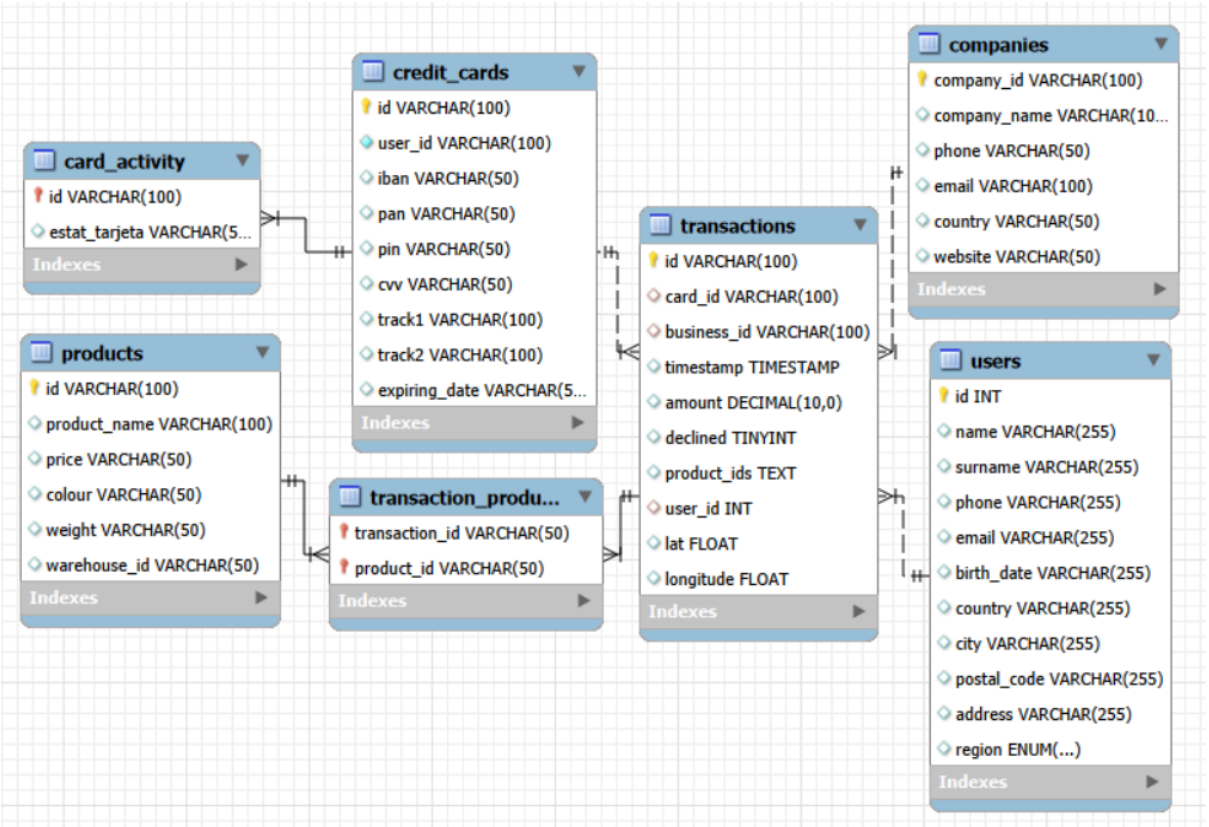
Result 9

Output

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 23 | 12:17:24 | SELECT * FROM transaction_products | 253391 row(s) returned |
| 24 | 12:17:55 | SELECT product_id, product_name, COUNT(product_id) AS total_ventas FROM transaction_products tp... | 100 row(s) returned |

Feta la part complicada, per resoldre l'exercici, ja només queda comptar la quantitat de repeticions per id de producte (product_id). Relacionem amb la taula productes mitjançant JOIN, per agafar els camps que ens puguin interessar, en aquest cas, el nom de producte, per fer més comprensible el resultat de la cerca.




En observar el diagrama final, veiem en aquest model de floc de neu, que tenim 7 taules relacionades, una d'elles a través d'una taula normalitzada (amb una clau primària depenent de les dues taules implicades), i una taula derivada, per la consulta sobre les targetes actives